Práctica 1

Generated by Doxygen 1.9.8

1 Test List 1

2 Data Structure Index 3

2.1 Data Structures . . . 3

3 File Index 5

3.1 File List . . . 5

4 Data Structure Documentation 7

4.1 Arg_hilos Struct Reference . . . 7

4.1.1 Detailed Description . . . 7

4.1.2 Field Documentation . . . 7

4.1.2.1 found_value . . . 7

4.1.2.2 max . . . 7

4.1.2.3 min . . . 8

4.1.2.4 target . . . 8

4.2 Logger_args Struct Reference . . . 8

4.2.1 Detailed Description . . . 8

4.2.2 Field Documentation . . . 8

4.2.2.1 id . . . 8

4.2.2.2 solution . . . 9

4.2.2.3 target . . . 9

4.2.2.4 validated . . . 9

4.2.2.5 votes . . . 9

4.2.2.6 wallets . . . 9

4.2.2.7 winner . . . 9

4.3 Miner_data Struct Reference . . . 9

4.3.1 Detailed Description . . . 10

4.3.2 Field Documentation . . . 10

4.3.2.1 n_threads . . . 10

4.3.2.2 rounds . . . 10

4.3.2.3 target . . . 10

5 File Documentation 11

5.1 logger.c File Reference . . . 11

5.1.1 Detailed Description . . . 12

5.1.2 Function Documentation . . . 12

5.1.2.1 logger() . . . 12

5.2 unit_tests/miner_test/logger.c File Reference . . . 12

5.2.1 Detailed Description . . . 13

5.2.2 Function Documentation . . . 14

5.2.2.1 logger() . . . 14

5.3 logger.h File Reference . . . 14

5.3.1 Detailed Description . . . 15

5.3.2 Function Documentation . . . 16
5.3.2.1 logger() . . . 16
5.4 logger.h . . . 16
5.5 unit_tests/miner_test/logger.h File Reference . . . 16
5.5.1 Detailed Description . . . 18
5.5.2 Function Documentation . . . 18
5.5.2.1 logger() . . . 18
5.6 logger.h . . . 18
5.7 miner.c File Reference . . . 19
5.7.1 Detailed Description . . . 20
5.7.2 Function Documentation . . . 20
5.7.2.1 calcular_solucion() . . . 20
5.7.2.2 comunicar_logger() . . . 21
5.7.2.3 minero() . . . 21
5.7.2.4 open_pipes() . . . 21
5.7.2.5 pow_seek() . . . 21
5.8 unit_tests/miner_test/miner.c File Reference . . . 22
5.8.1 Detailed Description . . . 23
5.8.2 Function Documentation . . . 23
5.8.2.1 calcular_solucion() . . . 23
5.8.2.2 comunicar_logger() . . . 23
5.8.2.3 minero() . . . 24
5.8.2.4 open_pipes() . . . 24
5.8.2.5 pow_seek() . . . 24
5.9 miner.h File Reference . . . 25
5.9.1 Detailed Description . . . 26
5.9.2 Function Documentation . . . 26
5.9.2.1 calcular_solucion() . . . 26
5.9.2.2 minero() . . . 27
5.9.2.3 open_pipes() . . . 27
5.9.2.4 pow_seek() . . . 27
5.10 miner.h . . . 27
5.11 unit_tests/miner_test/miner.h File Reference . . . 28
5.11.1 Detailed Description . . . 29
5.11.2 Function Documentation . . . 30
5.11.2.1 calcular_solucion() . . . 30
5.11.2.2 minero() . . . 30
5.11.2.3 open_pipes() . . . 30
5.11.2.4 pow_seek() . . . 31
5.12 miner.h . . . 31
5.13 mrush.c File Reference . . . 31
5.13.1 Detailed Description . . . 32

5.14 pow.h File Reference . . . 33
5.14.1 Detailed Description . . . 34
5.14.2 Macro Definition Documentation . . . 34
5.14.2.1 POW_LIMIT . . . 34
5.14.3 Function Documentation . . . 34
5.14.3.1 pow_hash() . . . 34
5.15 pow.h . . . 35
5.16 unit_tests/miner_test/pow.h File Reference . . . 35
5.16.1 Detailed Description . . . 36
5.16.2 Macro Definition Documentation . . . 37
5.16.2.1 POW_LIMIT . . . 37
5.16.3 Function Documentation . . . 37
5.16.3.1 pow_hash() . . . 37
5.17 pow.h . . . 37
5.18 unit_tests/pow_test/pow.h File Reference . . . 37
5.18.1 Detailed Description . . . 39
5.18.2 Macro Definition Documentation . . . 39
5.18.2.1 POW_LIMIT . . . 39
5.18.3 Function Documentation . . . 39
5.18.3.1 pow_hash() . . . 39
5.19 pow.h . . . 40
5.20 types.h File Reference . . . 40
5.20.1 Detailed Description . . . 42
5.20.2 Macro Definition Documentation . . . 42
5.20.2.1 die . . . 42
5.20.2.2 die_msg . . . 42
5.20.2.3 ERR . . . 43
5.20.2.4 FOUND . . . 43
5.20.2.5 MIN . . . 43
5.20.2.6 OK . . . 43
5.20.2.7 READ . . . 43
5.20.2.8 WRITE . . . 43
5.20.3 Typedef Documentation . . . 44
5.20.3.1 f32 . . . 44
5.20.3.2 f64 . . . 44
5.20.3.3 i32 . . . 44
5.20.3.4 i64 . . . 44
5.20.3.5 u32 . . . 44
5.20.3.6 u64 . . . 44
5.21 types.h . . . 45
5.22 unit_tests/miner_test/types.h File Reference . . . 45
5.22.1 Detailed Description . . . 47

5.22.2 Macro Definition Documentation . . . 47
5.22.2.1 die . . . 47
5.22.2.2 die_msg . . . 47
5.22.2.3 ERR . . . 48
5.22.2.4 FOUND . . . 48
5.22.2.5 MIN . . . 48
5.22.2.6 OK . . . 48
5.22.2.7 READ . . . 48
5.22.2.8 WRITE . . . 48
5.22.3 Typedef Documentation . . . 49
5.22.3.1 f32 . . . 49
5.22.3.2 f64 . . . 49
5.22.3.3 i32 . . . 49
5.22.3.4 i64 . . . 49
5.22.3.5 u32 . . . 49
5.22.3.6 u64 . . . 49
5.23 types.h . . . 50
5.24 unit_tests/pow_test/types.h File Reference . . . 50
5.24.1 Detailed Description . . . 52
5.24.2 Macro Definition Documentation . . . 52
5.24.2.1 die . . . 52
5.24.2.2 die_msg . . . 52
5.24.2.3 ERR . . . 53
5.24.2.4 FOUND . . . 53
5.24.2.5 MIN . . . 53
5.24.2.6 OK . . . 53
5.24.2.7 READ . . . 53
5.24.2.8 WRITE . . . 53
5.24.3 Typedef Documentation . . . 54
5.24.3.1 f32 . . . 54
5.24.3.2 f64 . . . 54
5.24.3.3 i32 . . . 54
5.24.3.4 i64 . . . 54
5.24.3.5 u32 . . . 54
5.24.3.6 u64 . . . 54
5.25 types.h . . . 55
5.26 unit_tests/miner_test/miner_test.c File Reference . . . 55
5.26.1 Detailed Description . . . 56
5.26.2 Function Documentation . . . 56
5.26.2.1 main() . . . 56
5.26.2.2 test1_open_pipes() . . . 57
5.26.2.3 test1_pow_seek() . . . 57

5.27 unit_tests/miner_test/miner_test.h File Reference . . . 57
5.27.1 Detailed Description . . . 58
5.27.2 Function Documentation . . . 58
5.27.2.1 test1_open_pipes() . . . 58
5.27.2.2 test1_pow_seek() . . . 58
5.28 miner_test.h . . . 59
5.29 unit_tests/pow_test/pow_test.c File Reference . . . 59
5.29.1 Detailed Description . . . 60
5.29.2 Function Documentation . . . 60
5.29.2.1 main() . . . 60
5.29.2.2 test1_pow() . . . 60
5.30 unit_tests/pow_test/pow_test.h File Reference . . . 61
5.30.1 Detailed Description . . . 61
5.30.2 Function Documentation . . . 62
5.30.2.1 test1_pow() . . . 62
5.31 pow_test.h . . . 62
5.32 test.h . . . 62
5.33 test.h . . . 63

Index **65**

Chapter 1

Test List

Global test1_open_pipes ()

Test open_pipes

Global test1_pow ()

Test pow function

Global test1_pow_seek ()

Test pow_seek

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

Arg_hilos
Estructura para manejo de argumentos de hilos Servira para guardar valores para busqueda de hash a traves de POW 7

Logger_args
Argumentos y datos del bloque minado Empleamos esta estructura para manejar los argumentos del registrador Son los argumentos que almacenara el registrador en el fichero 8

Miner_data
Estructura para manejo de argumentos de Minero Servira para guardar los datos relevantes para el minero 9

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

logger.c
Implementación de registrador 11
logger.h
Interfaz de registrador 14
miner.c
Implementación de minero 19
miner.h
Interfaz de mineros 25
mrush.c
Punto de entrada del programa 31
pow.h
Computation of the POW 33
types.h
Definición de tipos de datos globales 40
unit_tests/miner_test/logger.c
Implementación de registrador 12
unit_tests/miner_test/logger.h
Interfaz de registrador 16
unit_tests/miner_test/miner.c
Implementación de minero 22
unit_tests/miner_test/miner.h
Interfaz de mineros 28
unit_tests/miner_test/miner_test.c
Testea el modulo Miner 55
unit_tests/miner_test/miner_test.h
Declara los tests para el modulo miner 57
unit_tests/miner_test/pow.h
Computation of the POW 35
unit_tests/miner_test/test.h 62
unit_tests/miner_test/types.h
Definición de tipos de datos globales 45
unit_tests/pow_test/pow.h
Computation of the POW 37
unit_tests/pow_test/pow_test.c
Testea el modulo pow 59

unit_tests/pow_test/pow_test.h
Declara los tests para la funcion POW 61
unit_tests/pow_test/test.h 63
unit_tests/pow_test/types.h
Definición de tipos de datos globales 50

Chapter 4

Data Structure Documentation

4.1 Arg_hilos Struct Reference

Estructura para manejo de argumentos de hilos Servira para guardar valores para busqueda de hash a traves de POW.

```
#include <miner.h>
```

Data Fields

- u64 target
- u64 min
- u64 max
- volatile int * found_value

4.1.1 Detailed Description

Estructura para manejo de argumentos de hilos Servira para guardar valores para busqueda de hash a traves de POW.

4.1.2 Field Documentation

4.1.2.1 found_value

```
volatile int * Arg_hilos::found_value
```

Booleano de hallazgo de la solucion

4.1.2.2 max

```
u64 Arg_hilos::max
```

Valor maximo de busqueda

4.1.2.3 min

```
u64 Arg_hilos::min
```

Valor minimo de busqueda

4.1.2.4 target

```
u64 Arg_hilos::target
```

Objetivo de busqueda de funcion hash

The documentation for this struct was generated from the following files:

- miner.h
- unit_tests/miner_test/miner.h

4.2 Logger_args Struct Reference

Argumentos y datos del bloque minado Empleamos esta estructura para manejar los argumentos del registrador Son los argumentos que almacenara el registrador en el fichero.

```
#include <logger.h>
```

Data Fields

- u64 id
- u64 target
- u64 solution
- u64 votes
- u64 wallets
- pid_t winner
- i32 validated

4.2.1 Detailed Description

Argumentos y datos del bloque minado Empleamos esta estructura para manejar los argumentos del registrador Son los argumentos que almacenara el registrador en el fichero.

4.2.2 Field Documentation

4.2.2.1 id

```
u64 Logger_args::id
```

Numero de ronda

4.2.2.2 solution

```
u64 Logger_args::solution
```

Valor que resuelve el POW

4.2.2.3 target

```
u64 Logger_args::target
```

Objetivo de la ronda

4.2.2.4 validated

```
i32 Logger_args::validated
```

Booleano que representa validez de la solucion

4.2.2.5 votes

```
u64 Logger_args::votes
```

Numero de ronda

4.2.2.6 wallets

```
u64 Logger_args::wallets
```

En principio, id padre:numero ronda

4.2.2.7 winner

```
pid_t Logger_args::winner
```

ID Proceso padre

The documentation for this struct was generated from the following files:

- logger.h
- unit_tests/miner_test/logger.h

4.3 Miner_data Struct Reference

Estructura para manejo de argumentos de Minero Servira para guardar los datos relevantes para el minero.

```
#include <miner.h>
```

Data Fields

- u64 target
- u64 rounds
- u64 n_threads

4.3.1 Detailed Description

Estructura para manejo de argumentos de Minero Servira para guardar los datos relevantes para el minero.

4.3.2 Field Documentation

4.3.2.1 n_threads

```
u64 Miner_data::n_threads
```

Numero de hilos en los que separar carga de trabajo

4.3.2.2 rounds

```
u64 Miner_data::rounds
```

Numero maximo de rondas de busqueda

4.3.2.3 target

```
u64 Miner_data::target
```

Objetivo de busqueda de funcion hash

The documentation for this struct was generated from the following files:

- miner.h
- unit_tests/miner_test/miner.h

Chapter 5

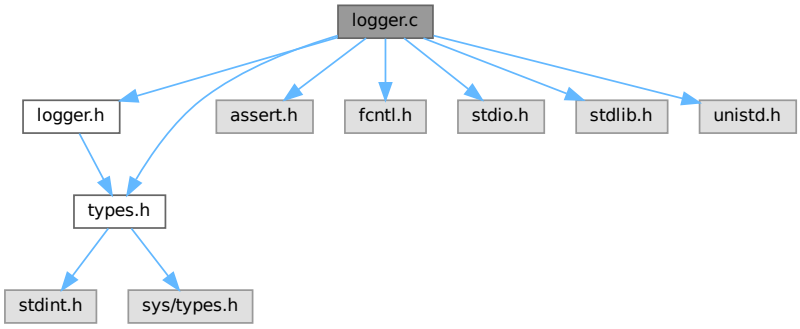
File Documentation

5.1 logger.c File Reference

Implementación de registrador.

```
#include "logger.h"
#include "types.h"
#include <assert.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

Include dependency graph for logger.c:



Functions

- i32 logger (i32 *miner_pipe, i32 *logger_pipe)

  Registra los datos del minero en un fichero.

5.1.1 Detailed Description

Implementación de registrador.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene la implementación y lógica de registro en fichero de disco.

Version

1.0

Date

2026-02-25

Copyright

(c) 2026 Author. All Rights Reserved.

5.1.2 Function Documentation

5.1.2.1 logger()

```
i32 logger (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Registra los datos del minero en un fichero.

- Se espera a leer informacion del minero a traves de miner_pipe. A continuación se escribe la informacion en el fichero con formato indicado en el enunciado. Por último se indica al minero que continue buscando la solucion de la siguiente ronda

Parameters

miner_pipe	Tuberia de escucha del minero
logger_pipe	Tuberia de escritura AL minero

Returns

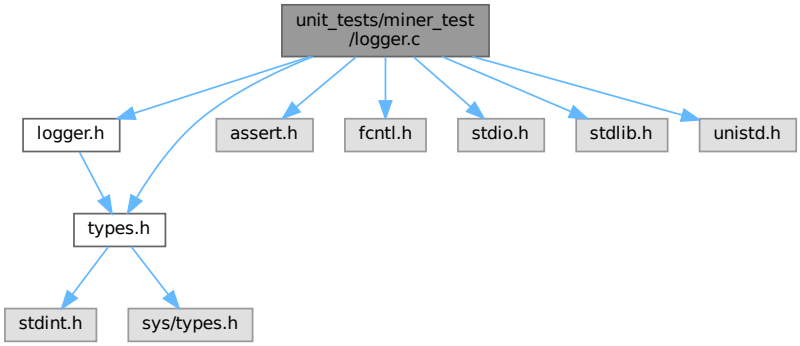
Devuelve EXIT_SUCCESS en caso de exito, EXIT_FAILURE en caso contrario

5.2 unit_tests/miner_test/logger.c File Reference

Implementación de registrador.

```
#include "logger.h"
#include "types.h"
#include <assert.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

Include dependency graph for logger.c:



Functions

- i32 logger (i32 *miner_pipe, i32 *logger_pipe)

  Registra los datos del minero en un fichero.

5.2.1 Detailed Description

Implementación de registrador.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene la implementación y lógica de registro en fichero de disco.

Version

1.0

Date

2026-02-25

Copyright

(c) 2026 Author. All Rights Reserved.

5.2.2 Function Documentation

5.2.2.1 logger()

```
i32 logger (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Registra los datos del minero en un fichero.

- Se espera a leer informacion del minero a traves de miner_pipe. A continuación se escribe la informacion en el fichero con formato indicado en el enunciado. Por último se indica al minero que continue buscando la solucion de la siguiente ronda

Parameters

miner_pipe	Tuberia de escucha del minero
logger_pipe	Tuberia de escritura AL minero

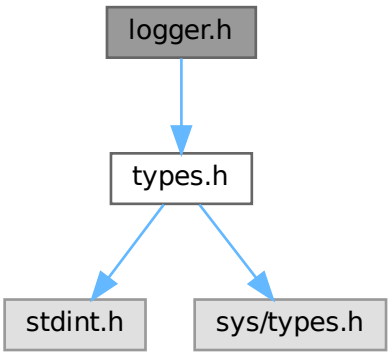
Returns

Devuelve EXIT_SUCCESS en caso de exito, EXIT_FAILURE en caso contrario

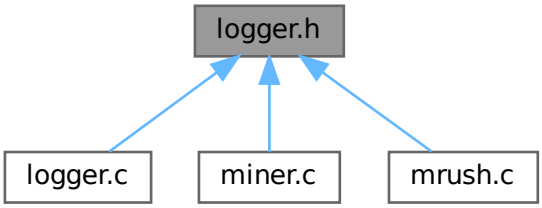
5.3 logger.h File Reference

Interfaz de registrador.

`#include "types.h"`
Include dependency graph for logger.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct Logger_args

  Argumentos y datos del bloque minado Empleamos esta estructura para manejar los argumentos del registrador Son los argumentos que almacenara el registrador en el fichero.

Functions

- i32 logger (i32 ∗miner_pipe, i32 ∗logger_pipe)

  Registra los datos del minero en un fichero.

5.3.1 Detailed Description

Interfaz de registrador.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene las funciones necesarias para inicializar el registrador y escribir el registro de soluciones en el archivo

Version

1.0

Date

2026-02-25

Copyright

(c) 2026 Author. All Rights Reserved.

5.3.2 Function Documentation

5.3.2.1 logger()

```
i32 logger (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Registra los datos del minero en un fichero.

- Se espera a leer informacion del minero a traves de miner_pipe. A continuación se escribe la informacion en el fichero con formato indicado en el enunciado. Por último se indica al minero que continue buscando la solucion de la siguiente ronda

Parameters

miner_pipe	Tuberia de escucha del minero
logger_pipe	Tuberia de escritura AL minero

Returns

Devuelve EXIT_SUCCESS en caso de exito, EXIT_FAILURE en caso contrario

5.4 logger.h

Go to the documentation of this file.

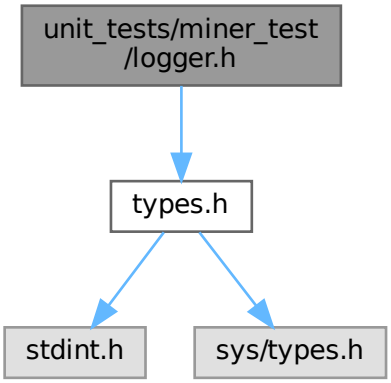
```
00001
00013 #ifndef _LOGGER_H
00014 #define _LOGGER_H
00015
00016 #include "types.h"
00017
00023 typedef struct {
00024   u64 id;
00025   u64 target;
00026   u64 solution;
00027   u64 votes;
00028   u64 wallets;
00029   pid_t winner;
00030   i32 validated;
00031 } Logger_args;
00032
00033 /* Dejamos pid_t(4bytes) y validated(4 Byte) para el final para minimizar
00034  * padding y alinear struct a 48 bytes. Podriamos usar tipos de menor tamaño
00035  * para validated, pero con 4 bytes conseguimos 0 bytes de padding */
00036
00049 i32 logger(i32 *miner_pipe, i32 *logger_pipe);
00050
00051 #endif
```

5.5 unit_tests/miner_test/logger.h File Reference

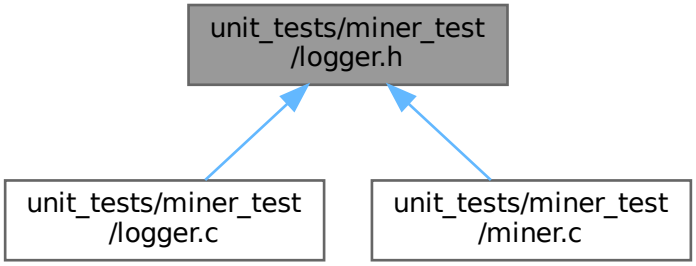
Interfaz de registrador.

```
#include "types.h"
```

Include dependency graph for logger.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct Logger_args

  Argumentos y datos del bloque minado Empleamos esta estructura para manejar los argumentos del registrador Son los argumentos que almacenara el registrador en el fichero.

Functions

- i32 logger (i32 ∗miner_pipe, i32 ∗logger_pipe)

  Registra los datos del minero en un fichero.

5.5.1 Detailed Description

Interfaz de registrador.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene las funciones necesarias para inicializar el registrador y escribir el registro de soluciones en el archivo

Version

1.0

Date

2026-02-25

Copyright

(c) 2026 Author. All Rights Reserved.

5.5.2 Function Documentation

5.5.2.1 logger()

```
i32 logger (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Registra los datos del minero en un fichero.

- Se espera a leer informacion del minero a traves de miner_pipe. A continuación se escribe la informacion en el fichero con formato indicado en el enunciado. Por último se indica al minero que continue buscando la solucion de la siguiente ronda

Parameters

miner_pipe	Tuberia de escucha del minero
logger_pipe	Tuberia de escritura AL minero

Returns

Devuelve EXIT_SUCCESS en caso de exito, EXIT_FAILURE en caso contrario

5.6 logger.h

Go to the documentation of this file.

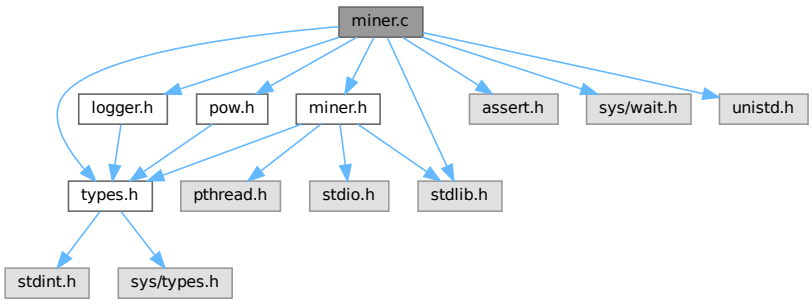
```
#ifndef _LOGGER_H
#define _LOGGER_H

#include "types.h"

typedef struct {
  u64 id;
  u64 target;
  u64 solution;
  u64 votes;
  u64 wallets;
  pid_t winner;
  i32 validated;
} Logger_args;

/* Dejamos pid_t(4bytes) y validated(4 Byte) para el final para minimizar
 * padding y alinear struct a 48 bytes. Podriamos usar tipos de menor tamaño
 * para validated, pero con 4 bytes conseguimos 0 bytes de padding */

i32 logger(i32 *miner_pipe, i32 *logger_pipe);

#endif
```

5.7 miner.c File Reference

Implementación de minero.

```
#include "miner.h"
#include "logger.h"
#include "pow.h"
#include "types.h"
#include <assert.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
```

Include dependency graph for miner.c:



Functions

- void comunicar_logger (i32 *miner_pipe, Logger_args *args)

  Manda un mensaje a logger Función auxiliar para minero.
- void open_pipes (i32 *miner_pipe, i32 *logger_pipe)

  Apertura de tuberias.
- void * pow_seek (void *arg)

  Busca el valor objetivo en un rango dado Funcion de calculo.

- u64 calcular_solucion (Miner_data *args)

  Crea los hilos y separa la tarea.
- void minero (Miner_data *args, i32 *miner_pipe, i32 *logger_pipe)

  Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

5.7.1 Detailed Description

Implementación de minero.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene la implementación de las funciones de cálculo del POW, así como la lógica del miner implementación de las funciones de cálculo del POW, así como la lógica del minero y el IPC entre minero y registrador

Version

1.0

Date

2026-02-12

Copyright

(c) 2026 Author. All Rights Reserved.

5.7.2 Function Documentation

5.7.2.1 calcular_solucion()

```
u64 calcular_solucion (
            Miner_data * args )
```

Crea los hilos y separa la tarea.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)

Returns

Devuelve la solucion para el POW con objetivo args->target

5.7.2.2 comunicar_logger()

```
void comunicar_logger (
            i32 * miner_pipe,
            Logger_args * args )
```

Manda un mensaje a logger Función auxiliar para minero.

Parameters

miner_pipe	Tuberia de escritura del minero al registrador
args	Estructura de argumentos del logger

5.7.2.3 minero()

```
void minero (
            Miner_data * args,
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)
miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero

5.7.2.4 open_pipes()

```
void open_pipes (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Apertura de tuberias.

Parameters

miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero En caso de error termina el programa con die()

5.7.2.5 pow_seek()

```
void * pow_seek (
            void * arg )
```

Busca el valor objetivo en un rango dado Funcion de calculo.

Parameters

arg	Puntero a lista de argumentos

Returns

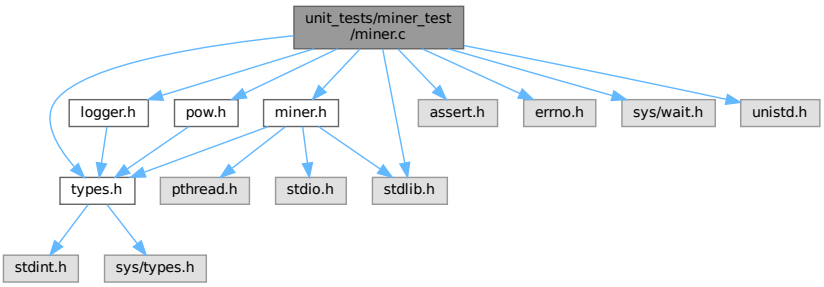
Devuelve puntero al numero que consigue el valor objetivo

5.8 unit_tests/miner_test/miner.c File Reference

Implementación de minero.

```
#include "miner.h"
#include "logger.h"
#include "pow.h"
#include "types.h"
#include <assert.h>
#include <errno.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
```

Include dependency graph for miner.c:



Functions

- void comunicar_logger (i32 ∗miner_pipe, Logger_args ∗args)

  Manda un mensaje a logger Función auxiliar para minero.
- void open_pipes (i32 ∗miner_pipe, i32 ∗logger_pipe)

  Apertura de tuberias.
- void ∗ pow_seek (void ∗arg)

  Busca el valor objetivo en un rango dado Funcion de calculo.
- u64 calcular_solucion (Miner_data ∗args)

  Crea los hilos y separa la tarea.
- void minero (Miner_data ∗args, i32 ∗miner_pipe, i32 ∗logger_pipe)

  Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

5.8.1 Detailed Description

Implementación de minero.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene la implementación de las funciones de cálculo del POW, así como la lógica del miner implementación de las funciones de cálculo del POW, así como la lógica del minero y el IPC entre minero y registrador

Version

1.0

Date

2026-02-12

Copyright

(c) 2026 Author. All Rights Reserved.

5.8.2 Function Documentation

5.8.2.1 calcular_solucion()

```
u64 calcular_solucion (
            Miner_data * args )
```

Crea los hilos y separa la tarea.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)

Returns

Devuelve la solucion para el POW con objetivo args->target

5.8.2.2 comunicar_logger()

```
void comunicar_logger (
            i32 * miner_pipe,
            Logger_args * args )
```

Manda un mensaje a logger Función auxiliar para minero.

Parameters

miner_pipe	Tuberia de escritura del minero al registrador
args	Estructura de argumentos del logger

5.8.2.3 minero()

```
void minero (
            Miner_data * args,
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)
miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero

5.8.2.4 open_pipes()

```
void open_pipes (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Apertura de tuberias.

Parameters

miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero En caso de error termina el programa con die()

5.8.2.5 pow_seek()

```
void * pow_seek (
            void * arg )
```

Busca el valor objetivo en un rango dado Funcion de calculo.

Parameters

arg	Puntero a lista de argumentos

Returns

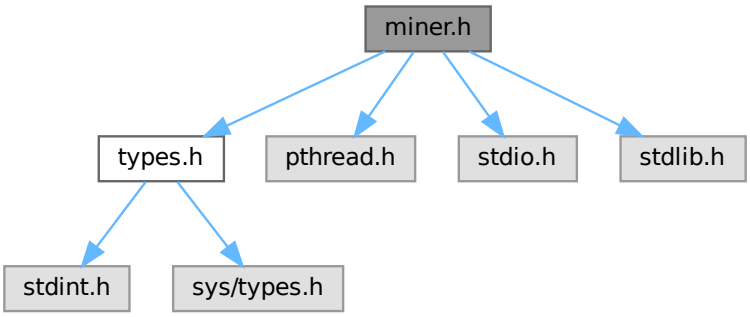
Devuelve puntero al numero que consigue el valor objetivo

5.9 miner.h File Reference

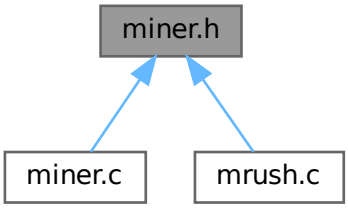
Interfaz de mineros.

```
#include "types.h"
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for miner.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct Arg_hilos

  Estructura para manejo de argumentos de hilos Servira para guardar valores para busqueda de hash a traves de POW.
- struct Miner_data

  Estructura para manejo de argumentos de Minero Servira para guardar los datos relevantes para el minero.

Functions

- void minero (Miner_data ∗args, i32 ∗miner_pipe, i32 ∗logger_pipe)

  Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.
- void ∗ pow_seek (void ∗arg)

  Busca el valor objetivo en un rango dado Funcion de calculo.
- u64 calcular_solucion (Miner_data ∗args)

  Crea los hilos y separa la tarea.
- void open_pipes (i32 ∗miner_pipe, i32 ∗logger_pipe)

  Apertura de tuberias.

5.9.1 Detailed Description

Interfaz de mineros.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene las declaración de funciones y estructuras necesarios para el minero y las tuberias así como las funciones de cálculo

Version

1.0

Date

2026-02-12

5.9.2 Function Documentation

5.9.2.1 calcular_solucion()

```
u64 calcular_solucion (
            Miner_data * args )
```

Crea los hilos y separa la tarea.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)

Returns

Devuelve la solucion para el POW con objetivo args->target

5.9.2.2 minero()

```
void minero (
            Miner_data * args,
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)
miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero

5.9.2.3 open_pipes()

```
void open_pipes (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Apertura de tuberias.

Parameters

miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero En caso de error termina el programa con die()

5.9.2.4 pow_seek()

```
void * pow_seek (
            void * arg )
```

Busca el valor objetivo en un rango dado Funcion de calculo.

Parameters

arg	Puntero a lista de argumentos

Returns

Devuelve puntero al numero que consigue el valor objetivo

5.10 miner.h

Go to the documentation of this file.

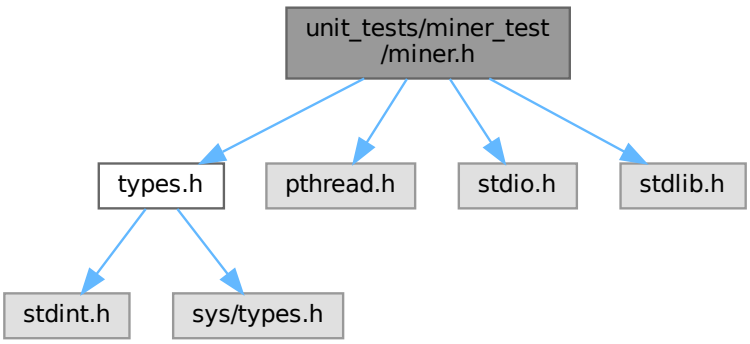
```
#ifndef _MINER_H
#define _MINER_H

#include "types.h"
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

/****************************** DATOS PUBLICOS ****************************/

typedef struct {
  u64 target;
  u64 min;
  u64 max;
  volatile int *found_value;
} Arg_hilos;

typedef struct {
  u64 target;
  u64 rounds;
  u64 n_threads;
} Miner_data;

/* Al ser estatica hay que definirla en .h porque todos los archivos que la usen
 * necesitan una copia del codigo, si estuviera en .c (opaca) habria error
 * puesto que no podrian emplear la cualidad de inline */

/****************************** DATOS PRIVADOS ****************************/

void minero(Miner_data *args, i32 *miner_pipe, i32 *logger_pipe);

void *pow_seek(void *arg);

u64 calcular_solucion(Miner_data *args);

void open_pipes(i32 *miner_pipe, i32 *logger_pipe);

#endif
```

5.11 unit_tests/miner_test/miner.h File Reference

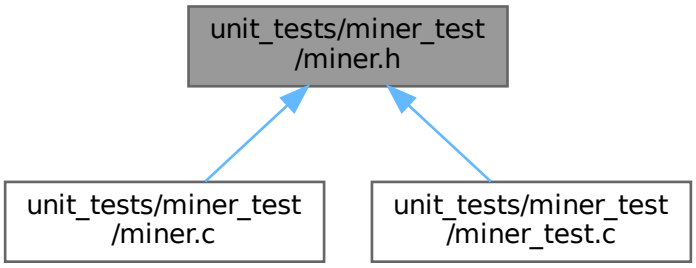
Interfaz de mineros.

```
#include "types.h"
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for miner.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct Arg_hilos

  Estructura para manejo de argumentos de hilos Servira para guardar valores para busqueda de hash a traves de POW.
- struct Miner_data

  Estructura para manejo de argumentos de Minero Servira para guardar los datos relevantes para el minero.

Functions

- void minero (Miner_data ∗args, i32 ∗miner_pipe, i32 ∗logger_pipe)

  Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.
- void ∗ pow_seek (void ∗arg)

  Busca el valor objetivo en un rango dado Funcion de calculo.
- u64 calcular_solucion (Miner_data ∗args)

  Crea los hilos y separa la tarea.
- void open_pipes (i32 ∗miner_pipe, i32 ∗logger_pipe)

  Apertura de tuberias.

5.11.1 Detailed Description

Interfaz de mineros.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene las declaración de funciones y estructuras necesarios para el minero y las tuberias así como las funciones de cálculo

Version

1.0

Date

2026-02-12

5.11.2 Function Documentation

5.11.2.1 calcular_solucion()

```
u64 calcular_solucion (
            Miner_data * args )
```

Crea los hilos y separa la tarea.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)

Returns

Devuelve la solucion para el POW con objetivo args->target

5.11.2.2 minero()

```
void minero (
            Miner_data * args,
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)
miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero

5.11.2.3 open_pipes()

```
void open_pipes (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Apertura de tuberias.

Parameters

miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero En caso de error termina el programa con die()

5.11.2.4 pow_seek()

```
void * pow_seek (
            void * arg )
```

Busca el valor objetivo en un rango dado Funcion de calculo.

Parameters

arg	Puntero a lista de argumentos

Returns

Devuelve puntero al numero que consigue el valor objetivo

5.12 miner.h

Go to the documentation of this file.

```
00001
00012 #ifndef _MINER_H
00013 #define _MINER_H
00014
00015 #include "types.h"
00016 #include <pthread.h>
00017 #include <stdio.h>
00018 #include <stdlib.h>
00019
00020 /****************************** DATOS PUBLICOS ****************************/
00021
00026 typedef struct {
00027   u64 target;
00028   u64 min;
00029   u64 max;
00030   volatile int *found_value;
00031 } Arg_hilos;
00032
00038 typedef struct {
00039   u64 target;
00040   u64 rounds;
00041   u64 n_threads;
00042 } Miner_data;
00043
00044 /* Al ser estatica hay que definirla en .h porque todos los archivos que la usen
00045  * necesitan una copia del codigo, si estuviera en .c (opaca) habria error
00046  * puesto que no podrian emplear la cualidad de inline */
00047
00048 /****************************** DATOS PRIVADOS ****************************/
00049
00060 void minero(Miner_data *args, i32 *miner_pipe, i32 *logger_pipe);
00061
00069 void *pow_seek(void *arg);
00070
00078 u64 calcular_solucion(Miner_data *args);
00079
00087 void open_pipes(i32 *miner_pipe, i32 *logger_pipe);
00088
00089 #endif
```

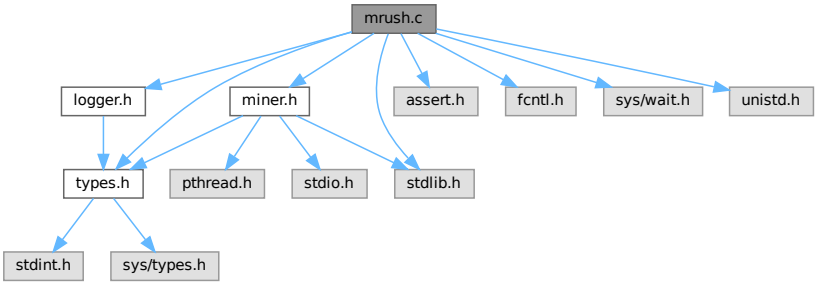
5.13 mrush.c File Reference

Punto de entrada del programa.

```
#include "logger.h"
#include "miner.h"
```

```
#include "types.h"
#include <assert.h>
#include <fcntl.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
```

Include dependency graph for mrush.c:



Functions

- int **main** (int argc, char *argv[ ])

5.13.1 Detailed Description

Punto de entrada del programa.

Author

Danyyil Shykerynets

- Contiene el main a ejecutar del programa

Version

1.0

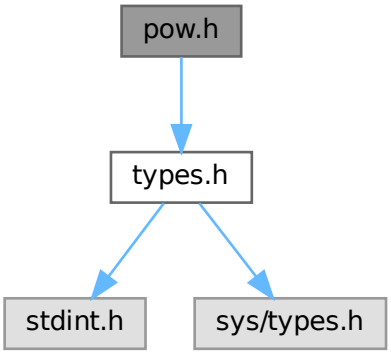
Date

2026-02-09

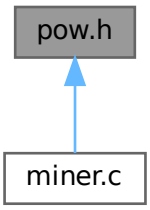
5.14 pow.h File Reference

Computation of the POW.

`#include "types.h"`
Include dependency graph for pow.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define POW_LIMIT 9997697

Functions

- u64 pow_hash (u64 x)

  Computes the following hash function: f(x) = (X x + Y) % P.

5.14.1 Detailed Description

Computation of the POW.

Author

SOPER teaching team.

Version

2.0

Date

2024-02-01

Copyright

Copyright (c) 2022

5.14.2 Macro Definition Documentation

5.14.2.1 POW_LIMIT

```
#define POW_LIMIT 9997697
```

Maximum number for the hash result.

5.14.3 Function Documentation

5.14.3.1 pow_hash()

```
u64 pow_hash (
            u64 x )
```

Computes the following hash function: f(x) = (X x + Y) % P.

Parameters

x	Argument of the hash function, x.

Returns

Result of the hash function, f(x).

5.15 pow.h

Go to the documentation of this file.

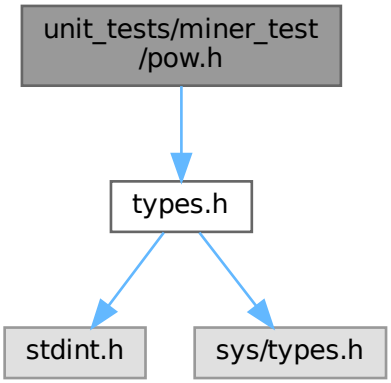
```
00001
00012 #ifndef _POW_H
00013 #define _POW_H
00014
00015 #include "types.h"
00016
00017 #define POW_LIMIT 9997697
00026 u64 pow_hash(u64 x);
00027
00028 #endif
```

5.16 unit_tests/miner_test/pow.h File Reference

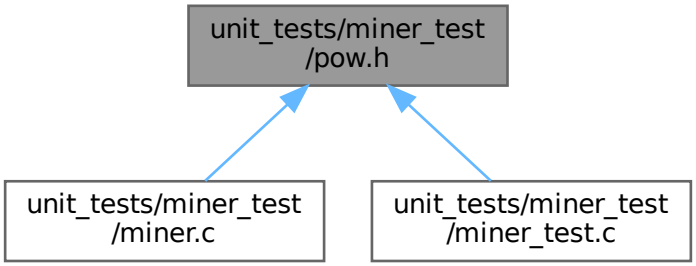
Computation of the POW.

`#include "types.h"`

Include dependency graph for pow.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define POW_LIMIT 9997697

Functions

- u64 pow_hash (long int x)

  Computes the following hash function: f(x) = (X x + Y) % P.

5.16.1 Detailed Description

Computation of the POW.

Author

SOPER teaching team.

Version

2.0

Date

2024-02-01

Copyright

Copyright (c) 2022

5.16.2 Macro Definition Documentation

5.16.2.1 POW_LIMIT

```
#define POW_LIMIT 9997697
```

Maximum number for the hash result.

5.16.3 Function Documentation

5.16.3.1 pow_hash()

```
u64 pow_hash (
            long int x )
```

Computes the following hash function: f(x) = (X x + Y) % P.

Parameters

x	Argument of the hash function, x.

Returns

Result of the hash function, f(x).

5.17 pow.h

Go to the documentation of this file.

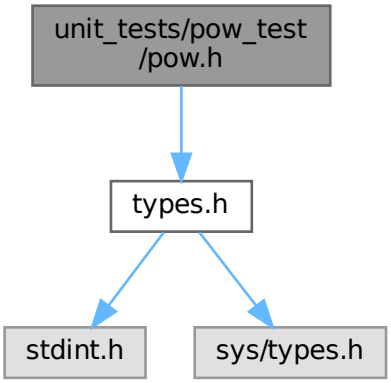
```
00001
00012 #ifndef _POW_H
00013 #define _POW_H
00014
00015 #include "types.h"
00016
00017 #define POW_LIMIT 9997697
00026 u64 pow_hash(long int x);
00027
00028 #endif
```

5.18 unit_tests/pow_test/pow.h File Reference

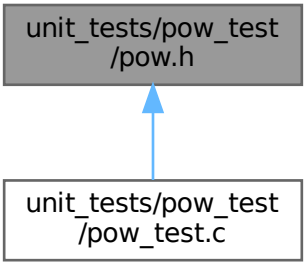
Computation of the POW.

```
#include "types.h"
```

Include dependency graph for pow.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define POW_LIMIT 9997697

Functions

- u64 pow_hash (u64 x)

  Computes the following hash function: f(x) = (X x + Y) % P.

5.18.1 Detailed Description

Computation of the POW.

Author

SOPER teaching team.

Version

2.0

Date

2024-02-01

Copyright

Copyright (c) 2022

5.18.2 Macro Definition Documentation

5.18.2.1 POW_LIMIT

```
#define POW_LIMIT 9997697
```

Maximum number for the hash result.

5.18.3 Function Documentation

5.18.3.1 pow_hash()

```
u64 pow_hash (
            u64 x )
```

Computes the following hash function: f(x) = (X x + Y) % P.

Parameters

x	Argument of the hash function, x.

Returns

Result of the hash function, f(x).

5.19 pow.h

Go to the documentation of this file.

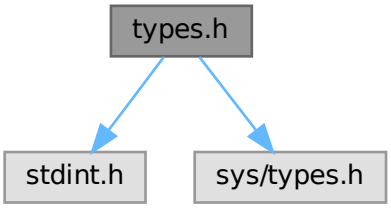
```
#ifndef _POW_H
#define _POW_H

#include "types.h"

#define POW_LIMIT 9997697
u64 pow_hash(u64 x);

#endif
```

5.20 types.h File Reference

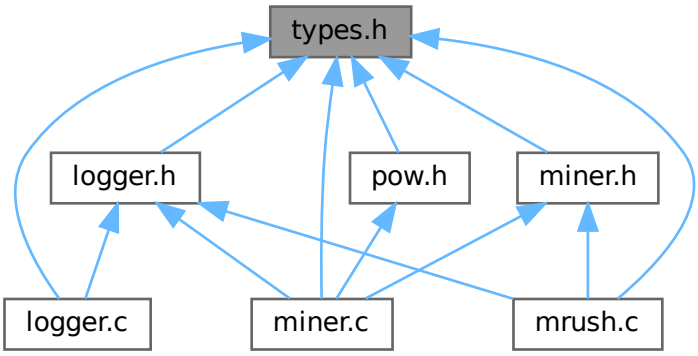
Definición de tipos de datos globales.

```
#include <stdint.h>
#include <sys/types.h>
```

Include dependency graph for types.h:



This graph shows which files directly or indirectly include this file:



Macros

Control de errores

- #define ERR -1
- #define OK !(ERR)
- #define die(msg)

  Imprimir error y terminar el programa.
- #define die_msg(msg)

  Termina el programa imprimiendo un mensaje lógico personalizado.

Extremos de tuberías (Pipes)

- #define READ 0
- #define WRITE 1

Banderas de estado

- #define FOUND 1

Utilidades generales

- #define MIN(a, b) ((a) < (b) ? (a) : (b))

  Devuelve el valor mínimo entre dos elementos.

Typedefs

Tipos de datos enteros y flotantes

- typedef int32_t i32
- typedef int64_t i64
- typedef uint32_t u32
- typedef uint64_t u64
- typedef float f32
- typedef double f64

5.20.1 Detailed Description

Definición de tipos de datos globales.

Author

Danyyil Shykerynets

- Este archivo contiene los alias de tipos de tamaño fijo (stdint) y las estructuras fundamentales compartidas entre el minero y el logger.

Version

1.0

Date

2026-02-09

5.20.2 Macro Definition Documentation

5.20.2.1 die

```
#define die(
            msg )
```

Value:

```
do {                                                    \
  perror(msg);                                          \
  exit(EXIT_FAILURE);                                   \
} while (0)
```

Imprimir error y terminar el programa.

Parameters

msg	Cadena con mensaje de error

5.20.2.2 die_msg

```
#define die_msg(
            msg )
```

Value:

```
do {                                                    \
  fprintf(stderr, "%s\n", msg);                         \
  exit(EXIT_FAILURE);                                   \
} while (0)
```

Termina el programa imprimiendo un mensaje lógico personalizado.

5.20.2.3 ERR

```
#define ERR -1
```

Código estándar de error para returns

5.20.2.4 FOUND

```
#define FOUND 1
```

Indica que un hilo ha encontrado solución

5.20.2.5 MIN

```
#define MIN(
            a,
            b ) ((a) < (b) ? (a) : (b))
```

Devuelve el valor mínimo entre dos elementos.

Parameters

a	Primer valor a comparar.
b	Segundo valor a comparar.

Returns

El menor de los dos valores.

5.20.2.6 OK

```
#define OK !(ERR)
```

Código de éxito (0)

5.20.2.7 READ

```
#define READ 0
```

Índice del descriptor de archivo para lectura

5.20.2.8 WRITE

```
#define WRITE 1
```

Índice del descriptor de archivo para escritura

5.20.3 Typedef Documentation

5.20.3.1 f32

```
typedef float f32
```

Float 32 bit. 7 dig. precision

5.20.3.2 f64

```
typedef double f64
```

Float 64 bit. 15 dig. precision

5.20.3.3 i32

```
typedef int32_t i32
```

Entero con signo 32 bit. [-2^31, 2^31 - 1]

5.20.3.4 i64

```
typedef int64_t i64
```

Entero con signo 64 bit. [-2^63, 2^63 - 1]

5.20.3.5 u32

```
typedef uint32_t u32
```

Entero sin signo 32 bit. [0, 2^32 - 1]

5.20.3.6 u64

```
typedef uint64_t u64
```

Entero sin signo 64 bit. [0, 2^64 - 1]

5.21 types.h

Go to the documentation of this file.

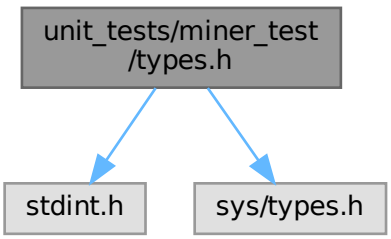
```
#ifndef _TYPES_H
#define _TYPES_H

#include <stdint.h>
#include <sys/types.h>

typedef int32_t i32;
typedef int64_t i64;
typedef uint32_t u32;
typedef uint64_t u64;
typedef float f32;
typedef double f64;
#define ERR -1
#define OK !(ERR)
#define die(msg)                                                    \
  do {                                                              \
    perror(msg);                                                    \
    exit(EXIT_FAILURE);                                             \
  } while (0)

#define die_msg(msg)                                                \
  do {                                                              \
    fprintf(stderr, "%s\n", msg);                                   \
    exit(EXIT_FAILURE);                                             \
  } while (0)
#define READ 0
#define WRITE 1
#define FOUND 1
#define MIN(a, b) ((a) < (b) ? (a) : (b))
#endif
```

5.22 unit_tests/miner_test/types.h File Reference

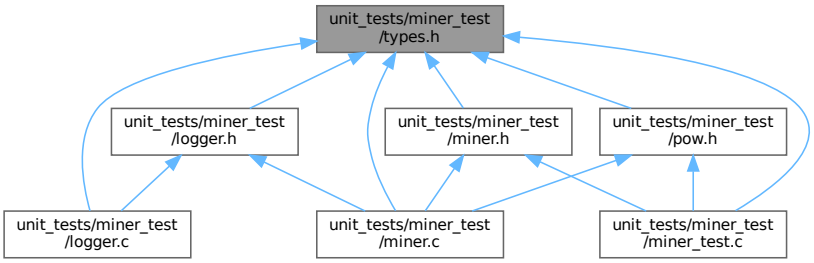
Definición de tipos de datos globales.

```
#include <stdint.h>
#include <sys/types.h>
```

Include dependency graph for types.h:



This graph shows which files directly or indirectly include this file:



Macros

Control de errores

- #define ERR -1
- #define OK !(ERR)
- #define die(msg)

  Imprimir error y terminar el programa.
- #define die_msg(msg)

  Termina el programa imprimiendo un mensaje lógico personalizado.

Extremos de tuberías (Pipes)

- #define READ 0
- #define WRITE 1

Banderas de estado

- #define FOUND 1

Utilidades generales

- #define MIN(a, b) ((a) < (b) ? (a) : (b))

  Devuelve el valor mínimo entre dos elementos.

Typedefs

Tipos de datos enteros y flotantes

- typedef int32_t i32
- typedef int64_t i64
- typedef uint32_t u32
- typedef uint64_t u64
- typedef float f32
- typedef double f64

5.22.1 Detailed Description

Definición de tipos de datos globales.

Author

Danyyil Shykerynets

- Este archivo contiene los alias de tipos de tamaño fijo (stdint) y las estructuras fundamentales compartidas entre el minero y el logger.

Version

1.0

Date

2026-02-09

5.22.2 Macro Definition Documentation

5.22.2.1 die

```
#define die(
            msg )
```

Value:

```
do {                                                    \
  perror(msg);                                          \
  exit(EXIT_FAILURE);                                   \
} while (0)
```

Imprimir error y terminar el programa.

Parameters

msg	Cadena con mensaje de error

5.22.2.2 die_msg

```
#define die_msg(
            msg )
```

Value:

```
do {                                                    \
  fprintf(stderr, "%s\n", msg);                         \
  exit(EXIT_FAILURE);                                   \
} while (0)
```

Termina el programa imprimiendo un mensaje lógico personalizado.

5.22.2.3 ERR

```
#define ERR -1
```

Código estándar de error para returns

5.22.2.4 FOUND

```
#define FOUND 1
```

Indica que un hilo ha encontrado solución

5.22.2.5 MIN

```
#define MIN(
            a,
            b ) ((a) < (b) ? (a) : (b))
```

Devuelve el valor mínimo entre dos elementos.

Parameters

a	Primer valor a comparar.
b	Segundo valor a comparar.

Returns

El menor de los dos valores.

5.22.2.6 OK

```
#define OK !(ERR)
```

Código de éxito (0)

5.22.2.7 READ

```
#define READ 0
```

Índice del descriptor de archivo para lectura

5.22.2.8 WRITE

```
#define WRITE 1
```

Índice del descriptor de archivo para escritura

5.22.3 Typedef Documentation

5.22.3.1 f32

```
typedef float f32
```

Float 32 bit. 7 dig. precision

5.22.3.2 f64

```
typedef double f64
```

Float 64 bit. 15 dig. precision

5.22.3.3 i32

```
typedef int32_t i32
```

Entero con signo 32 bit. [-2^31, 2^31 - 1]

5.22.3.4 i64

```
typedef int64_t i64
```

Entero con signo 64 bit. [-2^63, 2^63 - 1]

5.22.3.5 u32

```
typedef uint32_t u32
```

Entero sin signo 32 bit. [0, 2^32 - 1]

5.22.3.6 u64

```
typedef uint64_t u64
```

Entero sin signo 64 bit. [0, 2^64 - 1]

5.23 types.h

Go to the documentation of this file.

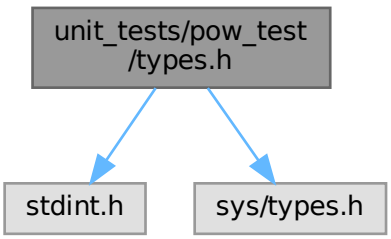
```
#ifndef _TYPES_H
#define _TYPES_H

#include <stdint.h>
#include <sys/types.h>

typedef int32_t i32;
typedef int64_t i64;
typedef uint32_t u32;
typedef uint64_t u64;
typedef float f32;
typedef double f64;
#define ERR -1
#define OK !(ERR)
#define die(msg)                                                    \
  do {                                                              \
    perror(msg);                                                    \
    exit(EXIT_FAILURE);                                             \
  } while (0)

#define die_msg(msg)                                                \
  do {                                                              \
    fprintf(stderr, "%s\n", msg);                                   \
    exit(EXIT_FAILURE);                                             \
  } while (0)
#define READ 0
#define WRITE 1
#define FOUND 1
#define MIN(a, b) ((a) < (b) ? (a) : (b))
#endif
```

5.24 unit_tests/pow_test/types.h File Reference

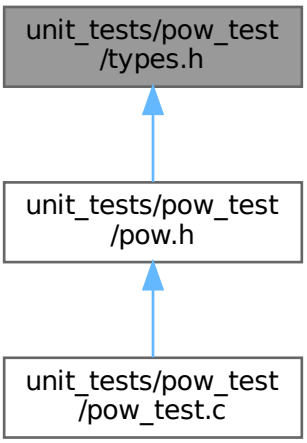
Definición de tipos de datos globales.

```
#include <stdint.h>
#include <sys/types.h>
```

Include dependency graph for types.h:



This graph shows which files directly or indirectly include this file:



Macros

Control de errores

- #define ERR -1
- #define OK !(ERR)
- #define die(msg)

  Imprimir error y terminar el programa.
- #define die_msg(msg)

  Termina el programa imprimiendo un mensaje lógico personalizado.

Extremos de tuberías (Pipes)

- #define READ 0
- #define WRITE 1

Banderas de estado

- #define FOUND 1

Utilidades generales

- #define MIN(a, b) ((a) < (b) ? (a) : (b))

  Devuelve el valor mínimo entre dos elementos.

Typedefs

Tipos de datos enteros y flotantes

- typedef int32_t i32
- typedef int64_t i64
- typedef uint32_t u32
- typedef uint64_t u64
- typedef float f32
- typedef double f64

5.24.1 Detailed Description

Definición de tipos de datos globales.

Author

Danyyil Shykerynets

- Este archivo contiene los alias de tipos de tamaño fijo (stdint) y las estructuras fundamentales compartidas entre el minero y el logger.

Version

1.0

Date

2026-02-09

5.24.2 Macro Definition Documentation

5.24.2.1 die

```
#define die(
            msg )
```

Value:

```
do {                                                    \
  perror(msg);                                          \
  exit(EXIT_FAILURE);                                   \
} while (0)
```

Imprimir error y terminar el programa.

Parameters

msg	Cadena con mensaje de error

5.24.2.2 die_msg

```
#define die_msg(
            msg )
```

Value:

```
do {                                                    \
  fprintf(stderr, "%s\n", msg);                         \
  exit(EXIT_FAILURE);                                   \
} while (0)
```

Termina el programa imprimiendo un mensaje lógico personalizado.

5.24.2.3 ERR

```
#define ERR -1
```

Código estándar de error para returns

5.24.2.4 FOUND

```
#define FOUND 1
```

Indica que un hilo ha encontrado solución

5.24.2.5 MIN

```
#define MIN(
            a,
            b ) ((a) < (b) ? (a) : (b))
```

Devuelve el valor mínimo entre dos elementos.

Parameters

a	Primer valor a comparar.
b	Segundo valor a comparar.

Returns

El menor de los dos valores.

5.24.2.6 OK

```
#define OK !(ERR)
```

Código de éxito (0)

5.24.2.7 READ

```
#define READ 0
```

Índice del descriptor de archivo para lectura

5.24.2.8 WRITE

```
#define WRITE 1
```

Índice del descriptor de archivo para escritura

5.24.3 Typedef Documentation

5.24.3.1 f32

```
typedef float f32
```

Float 32 bit. 7 dig. precision

5.24.3.2 f64

```
typedef double f64
```

Float 64 bit. 15 dig. precision

5.24.3.3 i32

```
typedef int32_t i32
```

Entero con signo 32 bit. [-2^31, 2^31 - 1]

5.24.3.4 i64

```
typedef int64_t i64
```

Entero con signo 64 bit. [-2^63, 2^63 - 1]

5.24.3.5 u32

```
typedef uint32_t u32
```

Entero sin signo 32 bit. [0, 2^32 - 1]

5.24.3.6 u64

```
typedef uint64_t u64
```

Entero sin signo 64 bit. [0, 2^64 - 1]

5.25 types.h

Go to the documentation of this file.

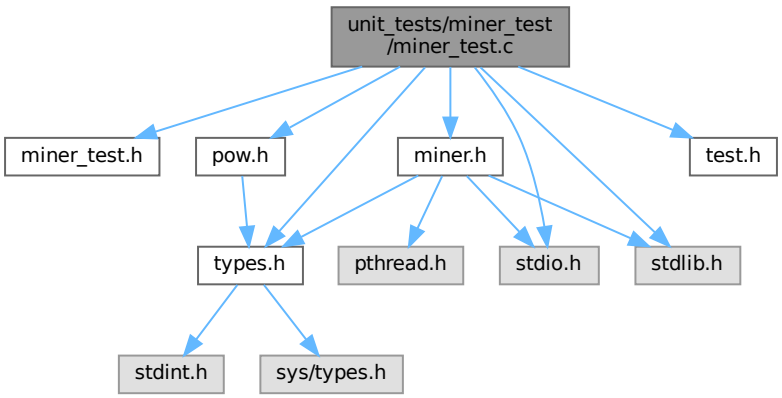
```
#ifndef _TYPES_H
#define _TYPES_H

#include <stdint.h>
#include <sys/types.h>

typedef int32_t i32;
typedef int64_t i64;
typedef uint32_t u32;
typedef uint64_t u64;
typedef float f32;
typedef double f64;
#define ERR -1
#define OK !(ERR)
#define die(msg)                                                    \
  do {                                                              \
    perror(msg);                                                    \
    exit(EXIT_FAILURE);                                             \
  } while (0)

#define die_msg(msg)                                                \
  do {                                                              \
    fprintf(stderr, "%s\n", msg);                                   \
    exit(EXIT_FAILURE);                                             \
  } while (0)
#define READ 0
#define WRITE 1
#define FOUND 1
#define MIN(a, b) ((a) < (b) ? (a) : (b))
#endif
```

5.26 unit_tests/miner_test/miner_test.c File Reference

Testea el modulo Miner.

```
#include "miner_test.h"
#include "miner.h"
#include "pow.h"
#include "test.h"
#include "types.h"
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for miner_test.c:



Macros

- #define **MAX_TESTS** 5

Functions

- int main (int argc, char ∗∗argv)

  Main function for Miner unit tests.
- void test1_open_pipes ()
- void **test2_open_pipes** ()
- void test1_pow_seek ()
- void **test2_pow_seek** ()
- void **test3_pow_seek** ()

5.26.1 Detailed Description

Testea el modulo Miner.

Author

Danyyil Shykerynets

Version

1.0

Date

01-03-2026 @Copyright (c) 2026 Author. All Rights Reserved.

5.26.2 Function Documentation

5.26.2.1 main()

```
int main (
            int argc,
            char ** argv )
```

Main function for Miner unit tests.

You may execute ALL or a SINGLE test 1.- No parameter -> ALL test are executed 2.- A number means a particular test (the one identified by that number) is executed

5.26.2.2 test1_open_pipes()

```
void test1_open_pipes ( )
```

Test Test open_pipes

Precondition

Pointers to file descriptor arrays

Postcondition

Initialized pipes in fd pointers

5.26.2.3 test1_pow_seek()

```
void test1_pow_seek ( )
```

Test Test pow_seek

Precondition

void∗ to thread_args

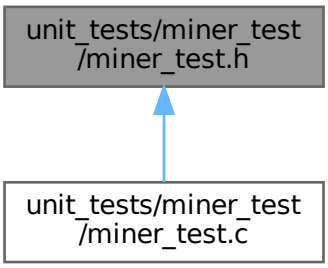
Returns

Pointer to POW solution

5.27 unit_tests/miner_test/miner_test.h File Reference

Declara los tests para el modulo miner.

This graph shows which files directly or indirectly include this file:



Functions

- void test1_open_pipes ()
- void **test2_open_pipes** ()
- void test1_pow_seek ()
- void **test2_pow_seek** ()
- void **test3_pow_seek** ()

5.27.1 Detailed Description

Declara los tests para el modulo miner.

Author

Danyyil Shykerynets

Version

1.0

Date

01-03-2025 @Copyright (c) 2026 Author. All Rights Reserved.

5.27.2 Function Documentation

5.27.2.1 test1_open_pipes()

```
void test1_open_pipes ( )
```

Test Test open_pipes

Precondition

Pointers to file descriptor arrays

Postcondition

Initialized pipes in fd pointers

5.27.2.2 test1_pow_seek()

```
void test1_pow_seek ( )
```

Test Test pow_seek

Precondition

void∗ to thread_args

Returns

Pointer to POW solution

5.28 miner_test.h

Go to the documentation of this file.

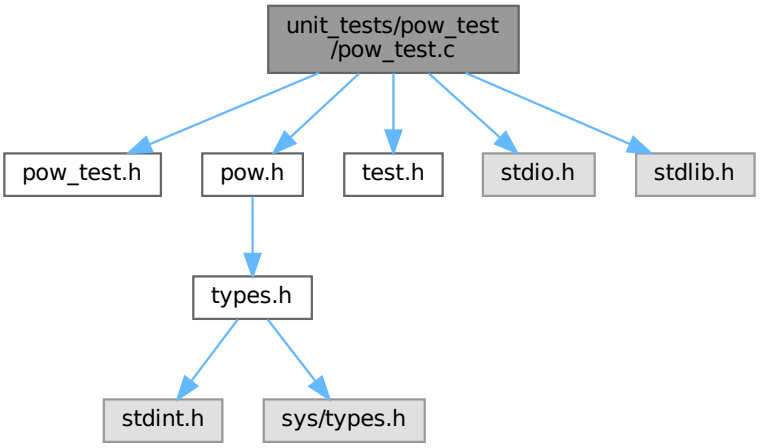
```
#ifndef _MINER_TEST_H
#define _MINER_TEST_H

void test1_open_pipes();
void test2_open_pipes();

void test1_pow_seek();
void test2_pow_seek();
void test3_pow_seek();

#endif
```

5.29 unit_tests/pow_test/pow_test.c File Reference

Testea el modulo pow.

```
#include "pow_test.h"
#include "pow.h"
#include "test.h"
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for pow_test.c:



Macros

- #define **MAX_TESTS** 3

Functions

- int main (int argc, char **argv)

  Main function for POW unit tests.
- void test1_pow ()
- void **test2_pow** ()
- void **test3_pow** ()

5.29.1 Detailed Description

Testea el modulo pow.

Author

Danyyil Shykerynets

Version

1.0

Date

01-03-2026 @Copyright (c) 2026 Author. All Rights Reserved.

5.29.2 Function Documentation

5.29.2.1 main()

```
int main (
            int argc,
            char ** argv )
```

Main function for POW unit tests.

You may execute ALL or a SINGLE test 1.- No parameter -> ALL test are executed 2.- A number means a particular test (the one identified by that number) is executed

5.29.2.2 test1_pow()

```
void test1_pow ( )
```

Test Test pow function

Precondition

Long int

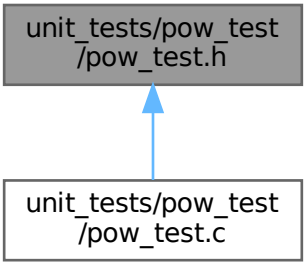
Postcondition

Hashed value

5.30 unit_tests/pow_test/pow_test.h File Reference

Declara los tests para la funcion POW.

This graph shows which files directly or indirectly include this file:



Functions

- void test1_pow ()
- void **test2_pow** ()
- void **test3_pow** ()

5.30.1 Detailed Description

Declara los tests para la funcion POW.

Author

Danyyil Shykerynets

Version

1.0

Date

01-03-2025 @Copyright (c) 2026 Author. All Rights Reserved.

5.30.2 Function Documentation

5.30.2.1 test1_pow()

```
void test1_pow ( )
```

Test Test pow function

Precondition

Long int

Postcondition

Hashed value

5.31 pow_test.h

Go to the documentation of this file.

```
#ifndef _POW_TEST_H
#define _POW_TEST_H

void test1_pow();
void test2_pow();
void test3_pow();

#endif
```

5.32 test.h

```
#ifndef _TEST_H
#define _TEST_H

#pragma GCC diagnostic ignored "-Wpedantic"

#ifndef NOCOLOR
#define KRED  "\x1B[31m"
#define KGRN  "\x1B[32m"
#define KYEL  "\x1B[33m"
#define KCYN  "\x1B[36m"
#define RESET "\033[0m"
#else
#define KRED
#define KGRN
#define KYEL
#define KCYN
#define RESET
#endif

#define PRINT_TEST_RESULT(x) do{                                    \
    __test_counter++;                                               \
    __pass = (x);                                                   \
    __test_passed = (__pass)? __test_passed + 1 : __test_passed;    \
    printf(KYEL "%s" RESET " line "  "%d " KCYN "%s" RESET ": %s\n",    \
       __FILE__, __LINE__ , __FUNCTION__,                           \
       ((!__pass) ? KRED "NOT PASS" RESET : KGRN "PASS" RESET));    \
  } while (0)

#define PRINT_PASSED_PERCENTAGE printf("Tests passed %d%%\n", ((__test_passed * 100) / __test_counter))

static int __test_counter = 0;
static int __test_passed  = 0;
static int __pass = 0;

#endif
```

5.33 test.h

```
#ifndef _TEST_H
#define _TEST_H

#pragma GCC diagnostic ignored "-Wpedantic"

#ifndef NOCOLOR
#define KRED "\x1B[31m"
#define KGRN "\x1B[32m"
#define KYEL "\x1B[33m"
#define KCYN "\x1B[36m"
#define RESET "\033[0m"
#else
#define KRED
#define KGRN
#define KYEL
#define KCYN
#define RESET
#endif

#define PRINT_TEST_RESULT(x)                                                    \
  do {                                                                          \
    __test_counter++;                                                           \
    __pass = (x);                                                               \
    __test_passed = (__pass) ? __test_passed + 1 : __test_passed;               \
    printf(KYEL "%s" RESET " line "                                             \
                "%d " KCYN "%s" RESET ": %s\n",                                 \
           __FILE__, __LINE__, __FUNCTION__,                                    \
           ((!__pass) ? KRED "NOT PASS" RESET : KGRN "PASS" RESET));            \
  } while (0)

#define PRINT_PASSED_PERCENTAGE                                                 \
  printf("Tests passed %d%%\n", ((__test_passed * 100) / __test_counter))

static int __test_counter = 0;
static int __test_passed = 0;
static int __pass = 0;

#endif
```

Index

Arg_hilos, 7
- found_value, 7
- max, 7
- min, 7
- target, 8

calcular_solucion
- miner.c, 20, 23
- miner.h, 26, 30

comunicar_logger
- miner.c, 20, 23

die
- types.h, 42, 47, 52

die_msg
- types.h, 42, 47, 52

ERR
- types.h, 42, 47, 52

f32
- types.h, 44, 49, 54

f64
- types.h, 44, 49, 54

FOUND
- types.h, 43, 48, 53

found_value
- Arg_hilos, 7

i32
- types.h, 44, 49, 54

i64
- types.h, 44, 49, 54

id
- Logger_args, 8

logger
- logger.c, 12, 14
- logger.h, 16, 18

logger.c, 11
- logger, 12, 14

logger.h, 14
- logger, 16, 18

Logger_args, 8
- id, 8
- solution, 8
- target, 9
- validated, 9
- votes, 9
- wallets, 9
- winner, 9

main
- miner_test.c, 56
- pow_test.c, 60

max
- Arg_hilos, 7

MIN
- types.h, 43, 48, 53

min
- Arg_hilos, 7

miner.c, 19
- calcular_solucion, 20, 23
- comunicar_logger, 20, 23
- minero, 21, 24
- open_pipes, 21, 24
- pow_seek, 21, 24

miner.h, 25
- calcular_solucion, 26, 30
- minero, 26, 30
- open_pipes, 27, 30
- pow_seek, 27, 30

Miner_data, 9
- n_threads, 10
- rounds, 10
- target, 10

miner_test.c
- main, 56
- test1_open_pipes, 56
- test1_pow_seek, 57

miner_test.h
- test1_open_pipes, 58
- test1_pow_seek, 58

minero
- miner.c, 21, 24
- miner.h, 26, 30

mrush.c, 31

n_threads
- Miner_data, 10

OK
- types.h, 43, 48, 53

open_pipes
- miner.c, 21, 24
- miner.h, 27, 30

pow.h, 33
- pow_hash, 34, 37, 39
- POW_LIMIT, 34, 37, 39

pow_hash
- pow.h, 34, 37, 39

POW_LIMIT
pow.h, 34, 37, 39
pow_seek
miner.c, 21, 24
miner.h, 27, 30
pow_test.c
main, 60
test1_pow, 60
pow_test.h
test1_pow, 62

READ
types.h, 43, 48, 53
rounds
Miner_data, 10

solution
Logger_args, 8

target
Arg_hilos, 8
Logger_args, 9
Miner_data, 10
Test List, 1
test1_open_pipes
miner_test.c, 56
miner_test.h, 58
test1_pow
pow_test.c, 60
pow_test.h, 62
test1_pow_seek
miner_test.c, 57
miner_test.h, 58
types.h, 40
die, 42, 47, 52
die_msg, 42, 47, 52
ERR, 42, 47, 52
f32, 44, 49, 54
f64, 44, 49, 54
FOUND, 43, 48, 53
i32, 44, 49, 54
i64, 44, 49, 54
MIN, 43, 48, 53
OK, 43, 48, 53
READ, 43, 48, 53
u32, 44, 49, 54
u64, 44, 49, 54
WRITE, 43, 48, 53

u32
types.h, 44, 49, 54
u64
types.h, 44, 49, 54
unit_tests/miner_test/logger.c, 12
unit_tests/miner_test/logger.h, 16, 18
unit_tests/miner_test/miner.c, 22
unit_tests/miner_test/miner.h, 28, 31
unit_tests/miner_test/miner_test.c, 55
unit_tests/miner_test/miner_test.h, 57, 59
unit_tests/miner_test/pow.h, 35, 37
unit_tests/miner_test/test.h, 62
unit_tests/miner_test/types.h, 45, 50
unit_tests/pow_test/pow.h, 37, 40
unit_tests/pow_test/pow_test.c, 59
unit_tests/pow_test/pow_test.h, 61, 62
unit_tests/pow_test/test.h, 63
unit_tests/pow_test/types.h, 50, 55

validated
Logger_args, 9
votes
Logger_args, 9

wallets
Logger_args, 9
winner
Logger_args, 9
WRITE
types.h, 43, 48, 53